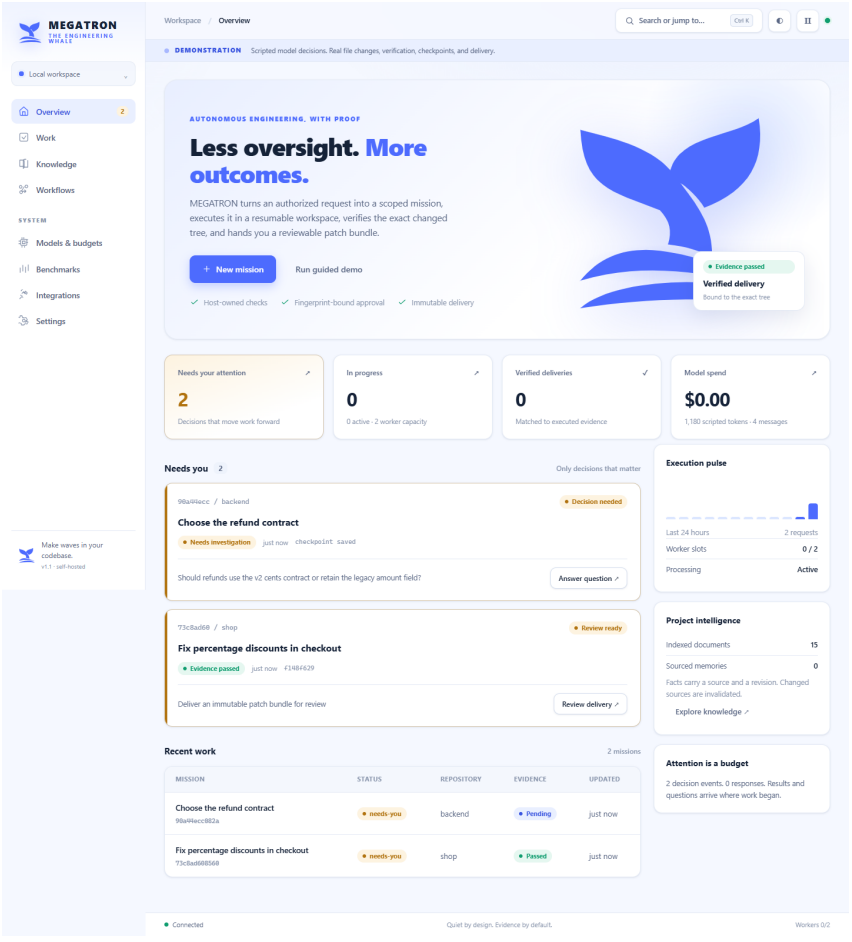
MEGATRON

THE ENGINEERING WHALE

PRODUCT · IMPLEMENTATION · UI / UX · OPERATIONS

Less oversight. More outcomes.

Evidence-first autonomous engineering. Version 1.1.0



REAL SOURCE CHANGES. EXECUTED CHECKS. REVIEWABLE DELIVERY.

The default demonstration is explicitly scripted; its execution and evidence are real.
Use a configured provider to measure live engineering performance on the same tasks.

The plan, in runnable form.

This guide combines product direction, the implemented capability ledger, the operating instructions, and the interface design. The original planning PDF is included separately as a historical reference; its targets are not presented as measured results.

1. Product and UI / UX

Source: docs/PRODUCT.md

2. Plan-to-code traceability

Source: docs/IMPLEMENTATION.md

3. How to use MEGATRON

Source: docs/USAGE.md

4. Architecture and recovery

Source: docs/ARCHITECTURE.md

5. Integrations

Source: docs/INTEGRATIONS.md

6. HTTP API

Source: docs/API.md

7. Testing and validation

Source: docs/TESTING.md

Product and UI / UX

The experience

MEGATRON's Mission Control turns attention economics into an interface. The first question is: **does any work need me?** The second is: **what evidence makes this result usable?** Technical internals stay inspectable without becoming the default reading burden.

The visual system is light-first and ocean-inspired: an airy blue-and-white canvas, a whale-tail MEGATRON mark, cobalt primary actions, amber decision states, green verified outcomes, and red failures. Typography uses system sans-serif text with monospaced IDs, commands, fingerprints, and output. Cards have consistent borders and spacing; modal elevation is reserved for concrete decisions. A complete dark theme retains the same semantic state colors.

Screens

1. **Overview:** attention-first mission cards, real execution counts, measured spend, work history, and project-intelligence context.

2. **Mission detail:** scope/mandate, factual timeline, actual before/after changes, independent evidence, resumable checkpoints, and delivery artifacts.

3. **Decision modal:** explicit consequence, change set, verified fingerprint, and approve/request-changes controls.

4. **Knowledge:** cited search results with source hashes and an import-relationship map.

5. **Workflows:** dependency-aware repository workstreams with separate mission links and node artifacts.

6. **Models and budgets:** capability routes, configured prices, and observed verification outcomes.

7. **Benchmarks:** transparent synthetic/live mode, expected outcomes, false-ready rejection, costs, and trace details.

8. **Integrations/settings:** sources, tools, validated skills, and revision-aware configuration.

Interaction rules

- `Ctrl+K` / `Cmd+K` opens the command palette.

- One structured question is answered through one clear form.

- Approve opens a reviewable decision; Escape closes a dialog without approving.

- Pausing processing preserves intake and stored work.

- Poll updates avoid replacing a focused editor or open dialog.

- Raw receipts expand progressively beneath concise states.

- Status labels accompany semantic colors.

- Visible focus rings, labeled controls, a skip link, and reduced-motion support are included.

- The mobile layout keeps all screens reachable through horizontally scrollable navigation; source tables scroll inside their own containers.

- Dark and light themes share the same semantic state model.

Product assets

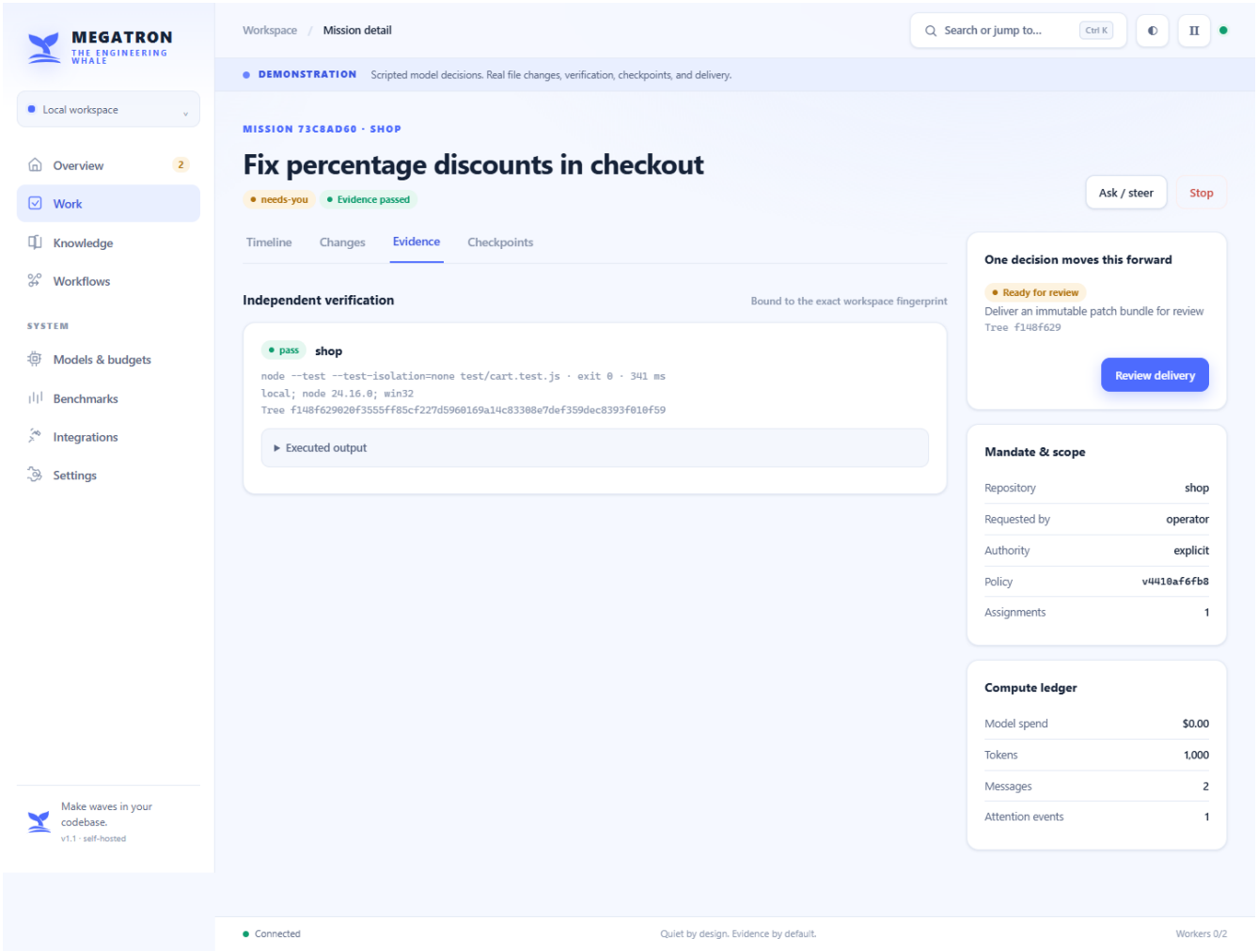
The shipped whale mark, wordmark, social card, screenshots, and mobile captures form one consistent MEGATRON brand system. Screenshots are taken from the actual running application, including real verification receipts and an approval-bound patch. The walkthrough video follows that same application. Demo mode is prominently labeled so a scripted provider is never represented as live intelligence. Asset licensing and adaptation details are recorded under `assets/brand/ATTRIBUTION.txt`.

The VS Code extension lists the same missions, renders script-free change review, and stores optional server tokens in SecretStorage. The mobile web manifest supplies a home-screen app surface backed by the same API.

Product evaluation

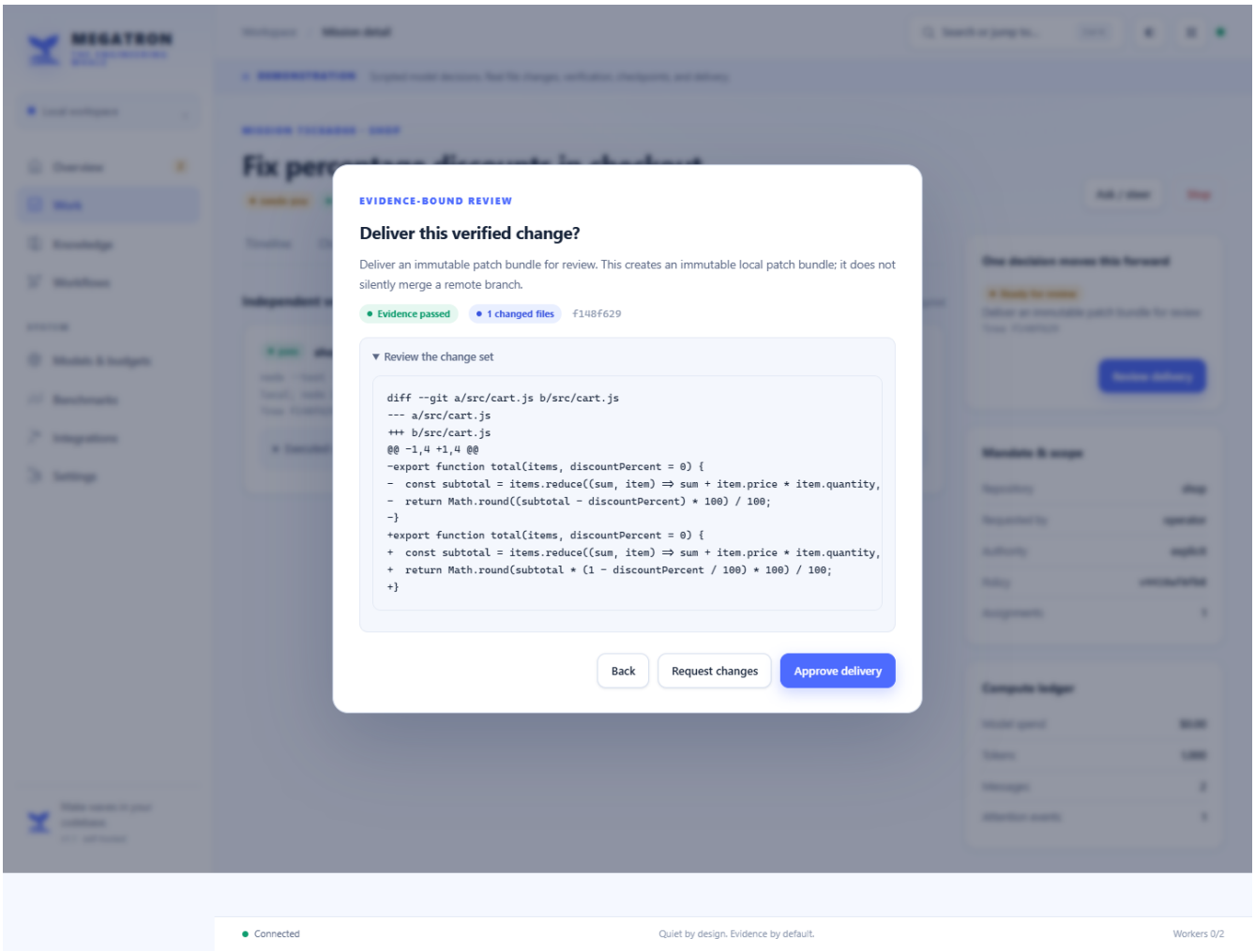
Measure verified completion, regressions, engineer interventions, time, and cost on equivalent tasks and budgets. Benchmark scripts establish harness correctness; configured-provider runs establish model-system performance. The original vision's percentage targets are goals to evaluate, not results to advertise.

Executed verification



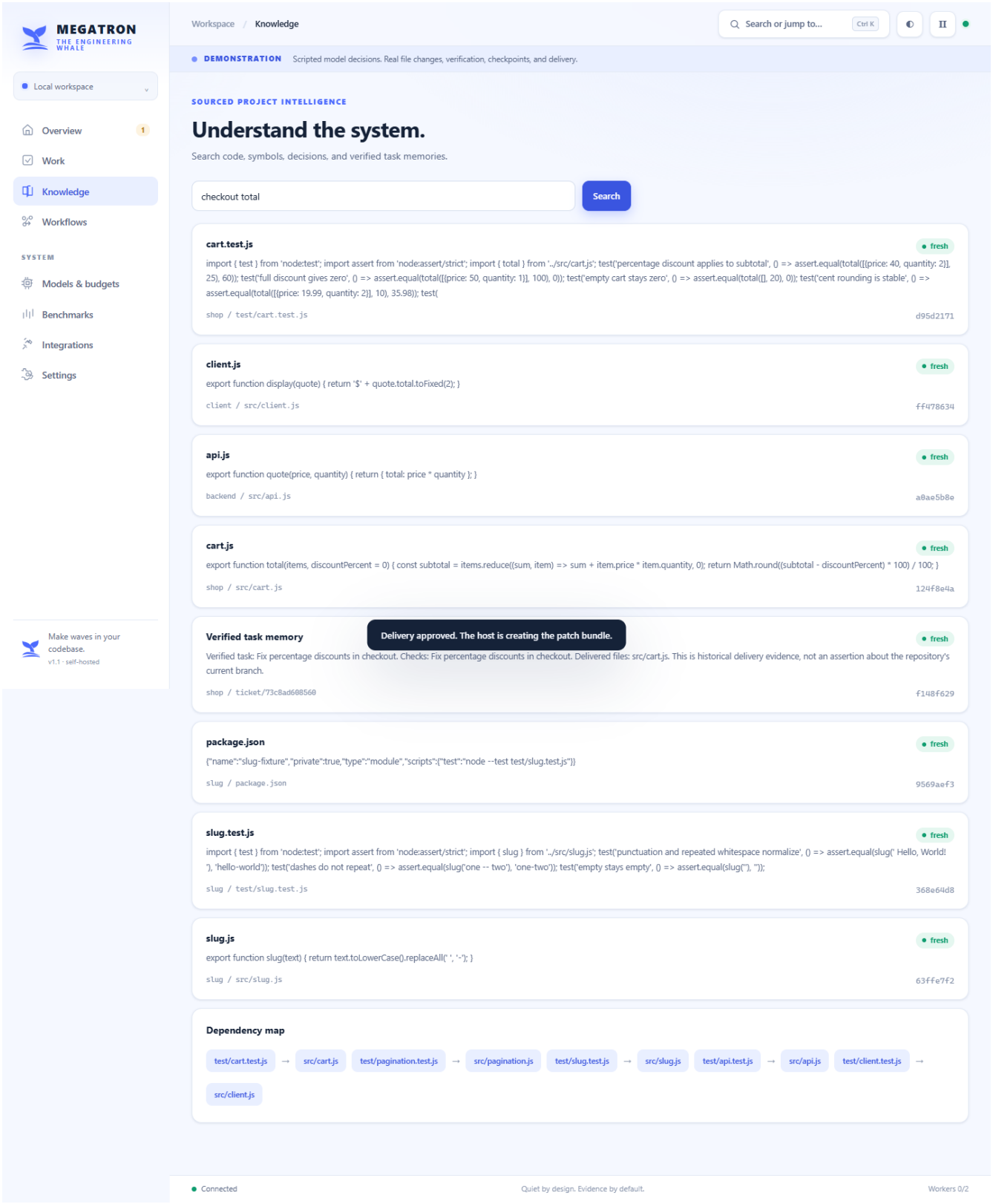
Captured from the running MEGATRON application. Demonstration mode is labeled; checks and artifacts are executed.

Evidence-bound review



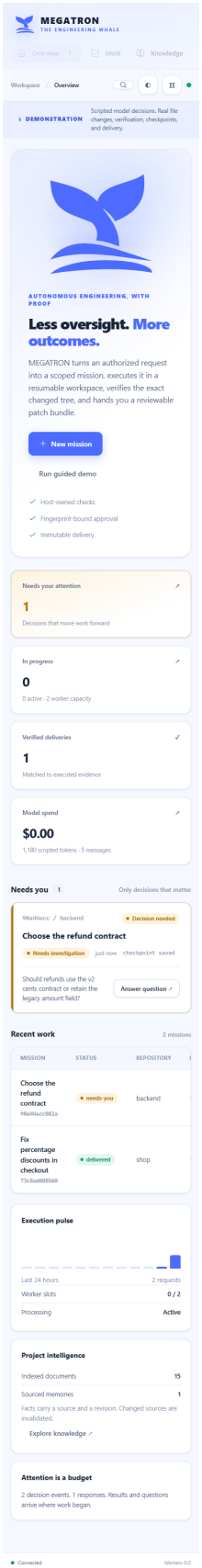
Captured from the running MEGATRON application. Demonstration mode is labeled; checks and artifacts are executed.

Source-cited project knowledge



Captured from the running MEGATRON application. Demonstration mode is labeled; checks and artifacts are executed.

Responsive mobile surface



Captured from the running MEGATRON application. Demonstration mode is labeled; checks and artifacts are executed.

Plan-to-code traceability

This is the implementation ledger for the original product plan. Each row names a runnable capability and its verification path. Planning targets in the original PDF are aspirations, not measured live-model results.

Plan item	Implementation	Verification
Benchmark/eval first	`src/bench/runner.ts`; fixtures; trace JSON; CI	Terminal benchmark; browser-triggered suite; adversarial false-ready case
Cost/attention	`src/ledger.ts`; console; `mg costs`	Idempotent messages; budget tests; measured per-case scorecards
Evidence readiness	`src/evidence.ts`; Engine verification; host-selected commands	Baseline exit 1 → fixed exit 0; stale tree refusal; unchanged trusted checks
Durable checkpoints	Worker transcripts; tool-run records; Engine recovery; retained workspaces	Nested transaction tests; effect coalescing; stopped-ticket and preparation tests
Approval gates	Delivery/application/publication/merge/deployment objects	Browser review/approve creates an actual bundle; stale head and fingerprint refusal
Sandboxes	Local adapter; constrained Docker command adapter	Path confinement, environment filtering, quoting, process-tree timeout; Docker requires an installed engine/image
Knowledge	FTS5, offline vectors, symbols/imports, sourced facts	Incremental index, dependents, source-hash invalidation, browser citation search
Work contracts	`src/workflows/dag.ts`; per-node file restrictions	Cycle/conflict checks; real backend/client dependencies and artifacts
Routing/speculation	`model-router.ts`; independent candidate runs	Capability selection; observed failures; actual failing/passing candidate comparison
Learning	`src/skills/learning.ts`; candidate → holdout → published	Training contamination refusal; clean two-task holdout; provenance retained
Multi-repo outcomes	Workflow scopes; dependency artifacts; rollout service	Backend/client fixture migration; declared health measurements and rollback receipts
Product surfaces	`mg`; web; VS Code extension; mobile manifest	CLI smoke; six isolated browser journeys; 390px layout; installable extension packaging

Additional improvements carried through

- Attention-first overview and quiet notification timing.
- Native browser investigation with screenshot, DOM, network, and console evidence.
- Factual event replay for debugging the agent itself.
- MCP stdio and Streamable HTTP clients; SHA-256-pinned local plugin packages.
- GitHub/Feishu source adapters and HMAC-signed external intake.
- Declared deployment-health thresholds and durable rollback outcomes.

- Core execution and default indexing without network services; optional local-model inference.

Operating prerequisites

Model API calls need a configured provider or local server. GitHub publication needs an authenticated GitHub CLI with repository access. Feishu needs its configured app and environment secret. Docker execution needs Docker and the prepared image. Browser investigation needs Playwright and an installed browser.

The default demo makes all scripted decisions explicit while using real source files, checks, checkpoints, gates, artifacts, and cost/attention records. Its measured scorecard validates those mechanics. Use configured-provider benchmarks to evaluate engineering quality and any competitive claim.

How to use MEGATRON

1. Runtime and first launch

Use Node.js 24+. There is no transpilation step. The CLI and console use standard Node modules; development checks and optional browser investigation use the dependencies installed by `npm ci`.

```
node bin/mg.js init
node bin/mg.js serve --demo
```

The default console URL is `http://127.0.0.1:3082`. `--port` changes it. `--state` selects an absolute or relative state directory. Defaults are under the current project's `.megatron` directory, not your C-drive home directory. Demonstrations use a separate demo state.

2. The demonstration

```
node demo/run-demo.mjs
```

This executes a checkout discount correction, captures red/green test evidence, approves and delivers a patch bundle, rejects a false completion claim, and compares an incorrect candidate with a passing candidate in separate workspaces. Output includes a timeline, summary, evidence receipts, and immutable patch artifacts under `demo/output`.

The console labels this mode clearly. Its scripted provider is useful for reproducible tests; it is not a substitute for a configured model on arbitrary requests.

3. Register an application repository

```
node bin/mg.js repo add shop "G:\src\shop" --verify "node --test test/cart.test.js"
```

Edit the versioned settings to declare additional checks:

```
{
  "repositories": {
    "shop": {
      "path": "G:/src/shop",
      "github": "owner/shop",
      "baseBranch": "main",
      "verify": [
        {"id":"tests","label":"Checkout regression","command":"node --test
test/cart.test.js","timeoutMs":120000,"regression":true}
      ],
      "trustedTests": ["test/cart.test.js", "package.json"]
    }
  }
}
```

`regression: true` requires an exit-1 failing baseline and a passing patched run. Use it for a reproducible bug, not a new-feature check that already passes on the base. Trusted checks must remain unchanged. Verification is host-selected; model text cannot replace a required command with an easier one.

4. Connect a model

```
$env:OPENAI_API_KEY = "your-key"
node bin/mg.js model add main --provider openai --model YOUR_MODEL --key-env OPENAI_API_KEY

$env:ANTHROPIC_API_KEY = "your-key"
node bin/mg.js model add reviewer --provider anthropic --model YOUR_MODEL --key-env
ANTHROPIC_API_KEY
```

For compatible hosted or local APIs, supply `--base-url http://127.0.0.1:11434` (or your provider endpoint). Endpoints can include `/v1`; MEGATRON avoids duplicating that segment. Native tool-call and tool-result identifiers are preserved for both API formats.

Use `--input-price` and `--output-price` to set USD per 1,000 tokens, or edit model settings. Cost reporting uses these configured prices. The default demo prices are zero; live-provider prices must match your provider's billing schedule.

The router filters required capabilities and considers observed verification outcomes, latency, and configured prices. Multiple routes let unsuccessful work move to a stronger eligible model. API keys remain environment-variable references and are not written into transcripts or settings.

5. Work on a mission

```
node bin/mg.js run "Fix the checkout percentage-discount bug" --repo shop
node bin/mg.js ls
node bin/mg.js show TICKET_ID
node bin/mg.js diff TICKET_ID
node bin/mg.js replay TICKET_ID
node bin/mg.js ask TICKET_ID "Use the existing cart-wide discount behavior"
node bin/mg.js approve TICKET_ID
node bin/mg.js deny TICKET_ID "Please cover the rounding boundary"
node bin/mg.js stop TICKET_ID
node bin/mg.js retry TICKET_ID
```

Use `--json` for structured output. When a server is running for the selected state, CLI actions use its API rather than opening a second SQLite writer. Without a daemon, commands run and drain the local runtime themselves.

Approval authorizes creation of an immutable patch bundle. It does not claim a remote merge occurred. A source edit after verification invalidates the approval. The bundle contains `changes.patch`, `manifest.json`, the changed-tree snapshot, and receipts.

6. Explicit delivery actions

After reviewing a delivered bundle, request a separately scoped action:

```
node bin/mg.js action TICKET_ID apply
node bin/mg.js approve TICKET_ID

node bin/mg.js action TICKET_ID publish-pr
node bin/mg.js approve TICKET_ID

node bin/mg.js action TICKET_ID merge
node bin/mg.js approve TICKET_ID
```

Local application checks that the original repository has not changed since the baseline. PR publication uses the configured repository's Git data API and checks the original hashes of changed files. Merge checks the current PR head, a covering approval, CI results, and the small-change limit. GitHub branch protection remains authoritative.

For deployment, select an installed integration tool. Supply `healthTool`, `rollbackTool`, and `maxValue` in the action payload to execute a measured rollout. A failed health measurement invokes the declared rollback and records that

outcome.

7. Execution boundaries and budgets

Local mode uses the operator's OS identity. Its workspace path checks, excluded secret files, filtered child environments, and bounded commands are useful controls; local mode is not OS isolation.

For container execution, set `executor.sandbox` to `container`, install Docker, and prepare the configured image:

```
docker pull node:24-bookworm-slim
```

The adapter uses a read-only container root, a scoped workspace mount, no network, dropped capabilities, process/memory/CPU limits, and the unprivileged image user. Missing Docker or an unavailable image fails the work; the adapter never switches back to local mode silently. Dependencies must already be available in the chosen execution environment.

Limits include worker count, steps, tokens, USD, wall time, and bounded automatic correction attempts. Timeouts kill process trees. Budget stops retain checkpoints and source changes for review or a later retry.

8. Project intelligence and reusable skills

```
node bin/mg.js knowledge search "checkout discount" --repo shop
node bin/mg.js knowledge impact shop src/cart.js
node bin/mg.js knowledge index shop
node bin/mg.js skill propose TICKET_ID
node bin/mg.js skill validate SKILL_ID
```

Search combines exact FTS5 matches with deterministic offline vectors. Symbols and import relationships provide impact context. Facts require a file/hash citation or a verified-ticket fingerprint; changed source citations become stale on re-indexing and are excluded from retrieval. Verified deliveries add historical task evidence, not unqualified assertions about today's branch.

Candidate skills retain their training-ticket provenance. Publication requires clean outcomes on separate holdout tasks. The default trial uses declared scripts; a live benchmark is necessary to measure model-quality improvements.

9. Coordinated repositories

```
node bin/mg.js workflow demo --demo --auto-approve
```

The explicit demo flag runs the backend-to-client contract migration with demo approvals. Custom graphs can be submitted through the API. Each node declares a repository, role, file contract, dependencies, and requested outcome. Cycles are refused, overlapping same-repository edits are serialized, and completed artifacts unlock dependents.

10. Benchmarks, console, IDE, and mobile

```
node bin/mg.js bench --demo --json
node bin/mg.js bench --live --json
npm.cmd run package
```

Install the generated VS Code `.vsix` with **Extensions → Install from VSIX**. Configure `megatron.url`; tokens use VS Code SecretStorage. The responsive console also supplies a web-app manifest for mobile home-screen installation.

For LAN access, set `MEGATRON_SERVER_TOKEN`, then run `serve --host 0.0.0.0`. The console asks for the token; it establishes a same-site HTTP-only session. Put externally reachable instances behind your usual TLS gateway. API clients can use the bearer token. The default server stays on loopback and rejects unrelated hosts and cross-origin mutations.

For a TLS reverse proxy, set `MEGATRON_PUBLIC_ORIGIN` to the exact public HTTPS origin and preserve its Host header. The server uses that configured origin for same-origin checks and sets Secure session cookies.

11. Recovery and offline execution

Stop with Ctrl+C. Restart against the same state directory. The runtime recovers leased effects, preserves workspaces, and resumes interrupted assignments from stored transcripts and tool receipts. A command whose external completion is uncertain is not blindly repeated.

The CLI, database, deterministic knowledge search, scripts, and fixtures work offline. A local OpenAI-compatible tool-calling model can supply real inference without an external provider. Remote sources and integrations remain optional.

Architecture and recovery

Flow



Composition and ownership

- `src/core/model.ts` and `ports.ts`: typed facts and external boundaries.

- `src/engine.ts`: synchronous domain commands; the only ticket/assignment writer.

- `src/store/store.ts`: SQLite aggregate records, FTS5, append-only events, and leased effects.

- `src/effects.ts`: durable external work after transaction commit; coalesced pending keys do not lose work queued during execution.

- `src/outbox.ts`: ordered, idempotent channel receipts and notification timing.

- `src/executor/executor.ts`: per-ticket workspaces, slots, provider calls, verification, and actual artifact creation.

- `src/app/runtime.ts`: composition root, recovery, scheduler, integration wiring, and administration actions.

- `src/app/server.ts`: shared JSON API, same-origin controls, events, and static console.

The core does not execute a shell, contact a model, or send a remote message. Commands transact facts and queue effects. Adapters report the external outcomes back to the Engine.

Evidence and delivery

Required verification commands are copied from operator-controlled repository settings into the Ticket. A worker's `ready` report queues a host verification pass. Receipts contain the command, criterion, actual exit code/output, environment, duration, and SHA-256 tree fingerprint.

Trusted acceptance files are compared with the untouched baseline. Red/green criteria require a failing baseline. An empty receipt set cannot certify a mission. Changed work invalidates previous evidence and gates. Delivery independently checks the fingerprint and writes a complete immutable bundle before the domain records a delivered outcome.

Checkpoints, tools, and restarts

Each model step stores its protocol transcript. Tool intentions and results have stable IDs; replay can reuse known completed results. An uncertain interrupted command returns a reconciliation instruction rather than automatically repeating an external effect. Workspace preparation preserves existing files. A process lock prevents two runtimes for the same state directory.

SQLite transactions support nested savepoints and post-commit callbacks. Leased running effects recover to the durable pending queue. Retry counters distinguish capacity deferrals from failures. Timed-out commands and MCP clients wait for process termination before releasing resources.

Intelligence and coordination

The knowledge index is separate from authorization. It stores file content, hashes, symbols, imports, vectors, and historical task facts. Retrieval returns citations and excludes stale facts. Its offline vector implementation can be replaced through an embedding function without changing the query interface.

Workflow nodes define repository scopes, roles, file contracts, and dependencies. Scheduling rejects cycles and delays overlapping file contracts. Every node still receives its own Ticket and Mandate. A shared workflow does not implicitly widen a worker's authority.

Speculative candidates run in independent workspaces and are judged by actual checks. The winner is copied into the mission workspace and independently re-verified before delivery. Skills are versioned workflow instructions that must pass separate holdout validation before publication.

Providers and integrations

The provider adapter speaks native OpenAI and Anthropic tool protocols. Model routes carry capabilities and prices; observed verification success and latency feed routing. Keys are resolved from named environment variables.

Sources keep source-native identities. GitHub comments use stable edit IDs and hidden outgoing markers. Feishu replies use stable UUIDs. Signed webhooks pass through the same domain gate. Pinned plugins and MCP expose actual schemas to workers. Deployment integrations can supply health checks and rollback functions.

Product surfaces

The CLI, console, VS Code extension, and mobile web surface share the management API. No surface owns a separate agent loop. The console prioritizes attention, then progress, then audit details. Full factual events remain available for replay and benchmark investigation.

Integrations

GitHub

Authenticate the CLI in your own terminal. Set `sources.github` to `{repository,account,cli?,pollSeconds}` and give the corresponding configured repository its `github` owner/name. Configure source-native GitHub logins under `policy.sources.github.allowed`; bot identities go under that source's `bots`. Identical names on different sources do not inherit each other's permissions.

GitHub comment polling uses a durable cursor with overlap. Stable event IDs absorb replay. Outgoing comments carry hidden idempotency markers. A published PR is tied to the changed-file baseline, branch, and head receipt. A merge is a separate action with head, approval, CI, and size checks.

Feishu

Configure `sources.feishu` with chat IDs, app ID, bot ID, and `appSecretEnv`. Configure the signed-webhook secret and deliver Feishu events to its webhook route. Mentions and direct chats are translated into the same Trigger model; app secrets are resolved from the environment. Replies use a stable UUID.

Use Feishu-native user IDs under `policy.sources.feishu.allowed`. Signed intake uses its own `policy.sources.webhook` permissions.

MCP

```
node bin/mg.js mcp add tools --command node --args '["G:/tools/my-mcp-server.mjs"]'
```

For Streamable HTTP, add an entry with `transport: "http"` and its `url` in settings. Restart to initialize configured connections. The client performs protocol initialization, discovers tool schemas, and invokes native tool calls. Tool names are namespaced by server ID.

Pinned plugins

A plugin manifest names an executable module and its SHA-256. Installing is an operator action. Modifying the module after installation refuses reloading.

```
{
  "id":"example",
  "name":"Example integration",
  "version":"1.0.0",
  "module":"runner.mjs",
  "sha256":"SHA256_OF_MODULE_BYTES",
  "tools":[{"name":"inspect","description":"Inspect the
workspace","inputSchema":{"type":"object","properties":{},"additionalProperties":false}}]
}
```

The executable reads `{tool,args,workspace}` from stdin and returns text/JSON on stdout. Nonzero exits are tool failures. Arguments are checked against the declared schema. Keep effects within the declared workspace. Deployment plugins should expose idempotent deploy, health, and rollback operations.

Native browser investigation

Workers can call `browser_inspect` on a local development URL. The optional Playwright adapter captures title, visible content, failing requests, console errors, and a screenshot under the workspace's excluded `.tmp` area. It uses Edge on Windows and an installed Chromium browser elsewhere.

Local and air-gapped inference

Use an OpenAI-compatible tool-calling endpoint on loopback. Local endpoints can omit an API key. SQLite, transcripts, default vectors, fixtures, and delivery artifacts remain local. Disable external sources and MCP connections for an offline deployment.

HTTP API

Default: `http://127.0.0.1:3082`. JSON request bodies are capped at 1 MB. Mutations use POST. Errors have `{ "error": "...", "code": "..." }` when a domain code exists. A token-protected server accepts `Authorization: Bearer TOKEN` or its same-site browser session.

Queries

Route	Result
`GET /api/status` or `/api/overview`	Mode, work, attention, workers, sources, ledger, and health
`GET /api/tickets`	Mission summaries
`GET /api/tickets/:id`	Scope, assignments, evidence, gates, delivery, and costs
`GET /api/tickets/:id/diff`	Actual baseline-to-workspace patch and changes
`GET /api/tickets/:id/timeline`	Persisted factual events
`GET /api/events?since=SEQ`	Server-sent events with cursor IDs
`GET /api/knowledge?q=QUERY&repository=ID`	Ranked, cited, fresh context
`GET /api/knowledge/graph`	Indexed import nodes and edges
`GET /api/workflows`	Dependency graphs and node states
`GET /api/models`	Routes, prices, capabilities, and observations
`GET /api/costs`	Total and per-mission compute/attention accounting
`GET /api/benchmarks`	Background benchmark status and scorecards
`GET /api/skills`	Candidate/published/rejected skills
`GET /api/integrations`	Sources, plugins, MCP servers, tool schemas
`GET /api/settings`	Current revision and history

Commands

```
POST /api/run
{"request":"Fix checkout discounts","repository":"shop"}

POST /api/demo
{"scenario":"shop"}

POST /api/tickets/ID/ask
{"text":"Use the v2 contract"}

POST /api/tickets/ID/approve
{"approvalId":"GATE_ID"}

POST /api/tickets/ID/deny
{"approvalId":"GATE_ID","note":"Please address rounding"}

POST /api/tickets/ID/stop
{"reason":"Superseded by another change"}

POST /api/tickets/ID/retry
{}

POST /api/tickets/ID/action
{"action":"apply"}
```

Actions create a separate gate. `publish-pr` and `merge` require the configured GitHub repository. Deployment can carry a payload such as:

```
{"action":"deploy","payload":{"tool":"preview__deploy","healthTool":"preview__health","rollb
ackTool":"preview__rollback","maxValue":2500,"args":{}}}
```

`POST /api/workflows` accepts `{title,nodes}` or `{demo:true}`. Nodes declare `id,name,repository,request,role,dependsOn,files` and optionally a declared demo `scenario`.

`POST /api/knowledge/index` accepts `{repository}`. `POST /api/benchmarks` starts an isolated background suite and returns its job ID. `POST /api/skills/propose` accepts `{ticketId,name?}`. `POST /api/skills/validate` accepts `{id}` and executes separate baseline/candidate trials.

`POST /api/settings` accepts `{value,revision}`. A stale revision is refused. Sandbox/source changes return `needsRestart`. `POST /api/plugins` accepts a local manifest path selected by the operator. `POST /api/session` establishes the browser session from `{token}`.

Signed webhooks

Configure `sources.webhooks.secretEnv`. Send an HMAC-SHA256 hex signature of the exact JSON bytes in `x-megatron-signature`.

```
{"id":"event-123","repository":"shop","actor":"allowed-source-
id","human":true,"addressed":true,"text":"Fix the failing checkout test"}
```

Use `POST /api/webhooks/intake`. Requests still pass through the allowed-list and mandate gate. Feishu events use `/api/webhooks/feishu`; configure the app, chat IDs, bot ID, and its source-secret environment references.

Testing and validation

The recorded Windows run passed **41 Node tests**, **6 isolated browser scenarios**, all **5 positive regression fixtures**, and the adversarial false-ready rejection. See `VERIFICATION.json` and `assets/benchmark-scorecard.json` for the observed local results and their scope.

```
npm.cmd ci
npm.cmd run typecheck
npm.cmd test
npm.cmd run test:e2e
node demo/run-demo.mjs
node bin/mg.js bench --demo --json
```

The Node suite covers transactions/savepoints, post-commit callbacks, durable effect coalescing, capacity deferral, native provider protocols, malformed tool arguments, execution boundaries, child environment isolation, process-tree timeouts, readiness, stale approval refusal, budgets, structured questions, mandate withdrawal, MCP, pinned plugins, source translation, PR head refusal, indexing/freshness/impact, dependency graphs, model routing, skill holdouts, and rollout/rollback receipts.

Playwright exercises actual browser journeys: review evidence and approve a real patch bundle; answer a structured worker question; keyboard navigation and cited search; a 390px mobile layout; all console routes; the browser-triggered regression scoreboard; and the worker's native browser investigator. `npm run test:e2e` gives every journey a fresh daemon, database, and browser context so long-running tasks cannot pollute later results.

The benchmark uses five independently checked repository fixes and one false-completion claim. It captures scorecards and factual traces. Failures are inspectable artifacts rather than prompt-only judgments.

Windows testing uses the installed Edge channel. Linux/macOS use installed Chromium. Docker is an external execution prerequisite; exercising its actual engine/image requires that environment. GitHub and Feishu API contracts have deterministic checks, while authenticated remote delivery requires your accounts and repository access. Real model protocol tests run against local HTTP fixtures; live quality benchmarking requires a configured provider.

`npm run assets` captures screenshots and a walkthrough video. Install Playwright's FFmpeg helper first if needed. `npm run package` emits the complete source release and VS Code package with checksums.